```
        org 0
        mov sp, #0x10000
        mov r1, #GPIO_1
        mov r3, #GPIO_2
```

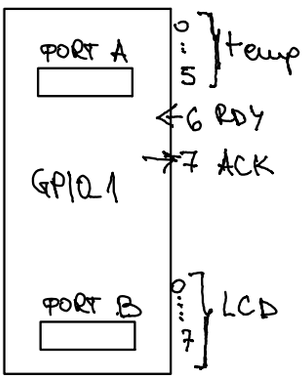
→ inicijalizacija stack pointera

↳ pokazivači na GPIO_1 i GPIO_2

INICIJALIZACIJA:

```
INIT    ldr r1, GPIO_1
        ldr r3, GPIO_2
```

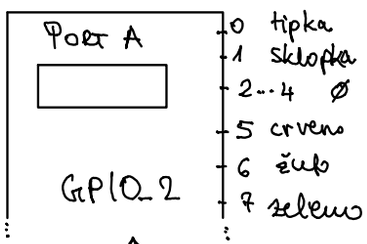
↳ loadaj podatke s tih adresa




```
        ;GPIO1 A        izlazi  ulazi
        mov r2, #0b10000000
        str r2, [r1, #0x8]
        ;GPIO1 B        izlazi
        mov r2, #0b00000000
        str r2, [r1, #0xC]

        ;GPIO2 A        izlazi  ulazi
        mov r2, #0b11100000
        str r2, [r3, #0x8]
```

postavljanje ulaza i izlaza




```
;pritisak i čekanje otpuštanja
loop1   ldr r5, [r3, #0]
        tst r5, #0b00000001
        bne loop2
        b loop1
```

→ port A od GPIO2 ;bezuvjetno čitaj stanje tipki

;ispitivanje bita 0

↳ je li uključena tipka (ako je 0 na najmanjem = TRUE)

NIJE Ø

```
loop2   ldr r5, [r3, #0]
        tst r5, #0b00000001
        beq temper
        b loop2
```

→ je li -||-

→ Ø je → idi dalje

trigger

```
;očitaj i spremi temp
temper
start   orr r5, r5, #0b10000000
        str r5, [r1, #0]
```

pokreni

→ simuliraj

- pokreni mjerenje
- digni ACK (bit 7) u 1

```
cekaj   ldr r5, [r1, #0]
        ands r5, r5, #0b01000000
        beq cekaj
```

čeka spremnost temp uređaja

- 8-bit registar podataka od GPIO1

→ možda još bolje TST R0, #0b01000000

↳ maskiraj status ready → ako je na 6. bitu 1 → idi dalje jer je READY

```
citaj   ldr r5, [r1, #0]
        and r5, r5, #0b00111111
        str r5, LAST
```

čitaj temp

→ čita samo najmanjih 5 bitova

....> spremi očitanje u LAST

```
end     bic r5, r5, #0b10000000
        str r5, [r1, #0]
```

kraj

→ makni taj 7. bit koji smo dodali za start (vrati ACK u nulu) → to smo jedino nešto mijenjali!

```
;je li temperatura H ili L
        ldr r4, LAST
```

→ u R4 loadaj iz LAST

```
HiliL   cmp r4, #19
        movhi r6, #0x48
        movle r6, #0x4C
```

usporedi sa 19

ASCII H → ako je > 19 → u r6 stavi H

ASCII L → ako je ≤ 19 → u r6 stavi L

```
prikaz   mov r0, #0x0D   → briši ulazne registre
         bl LCDWR        ← neka to bude znak
         mov r0, r6
         bl LCDWR
         mov r0, #0x0A   → kopiraj u prikazne
         bl LCDWR

         b loop1
```

nakon svake promjene znaka treba LCDWR kako bi se primjenila promjena

r0 = znak
r1 = adresa DR

šalje znak

```
LCDWR    stmfd r13!, {r0}      → spremi na stog da ne izgubimo od dr
         and r0, r0, #0x7f     ; postavi bit 7 u nulu (za svaki slučaj,
                               ; jer u r0 je tu mogla biti 1) i
                               ; pošalji znak
```

PB_DR

strobe

```
         strb r0, [r1, #4]     → pošalji znak na LCD
dižu impuls  ← orr r0, r0, #0x80   ; postavi bit 7 u jedan (podigni impuls)
         strb r0, [r1, #4]     → —"—
spusti impuls ← and r0, r0, #0x7f  ; postavi bit 7 u nulu (spusti impuls)
         strb r0, [r1, #4]     — " —
```

znak poslan i može se obraditi

```
vrati original sadržaj ← ldmfd r13!, {r0}
         mov pc, lr            ; povratak

kraj     swi      0x0123456

GPIO_1   dw       0xffff0f00
GPIO_2   dw       0xffff0b00

LAST     ds       1             ;spremamo 1 temp
```

↳ lokacija za spremanje temperature (1 bajt)

```
;- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
;Svaki puta kad se pritisne tipka, treba pročitati temperaturu s
;temperaturnog uređaja. Za svaki slučaj, nakon što ste prepoznali
;da je tipka pritisnuta pričekajte da se tipka otpusti.

;Zatim treba temperaturu spremiti u lokaciju LAST. Nakon toga treba
;na LCD ispisati „H“ (high) ako je temperatura strogo veća od 19
;ili „L“ (low) u suprotnom slučaju. Za slanje znaka na LCD
;upotrijebite funkciju LCDWR s predavanja (parametri: R0 je znak,
;a R1 je adresa LCD-a).

;Glavni program radi beskonačno.
```

LCDWR:

1. spremi „H“ u niksač
2. očisti niksač od ikakvih dodatnih znakova
3. stavi ga pred vrata LCD-a
4. pozovi zvono
5. prestani zvoniti
6. uzmi niksač
7. vrati se di si bio